Localization for Autonomous Navigation

Chapter 5. Multivariate Gaussians

Dept. Intelligent Systems & Robots

Prof. SungTae Moon





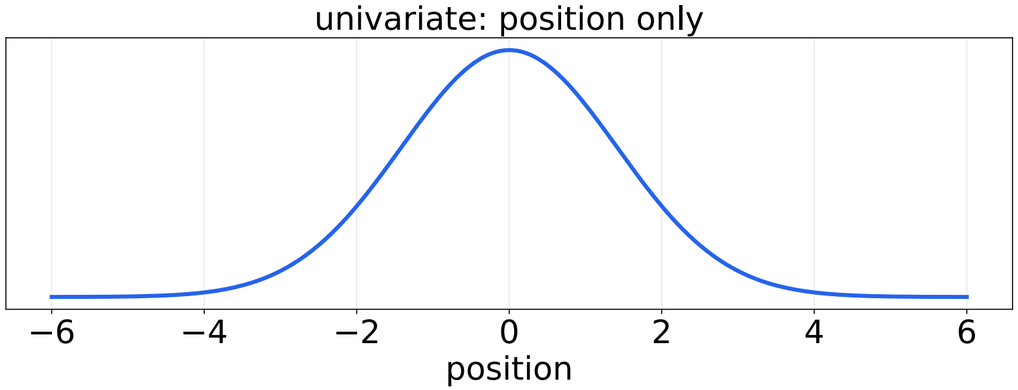
Contents

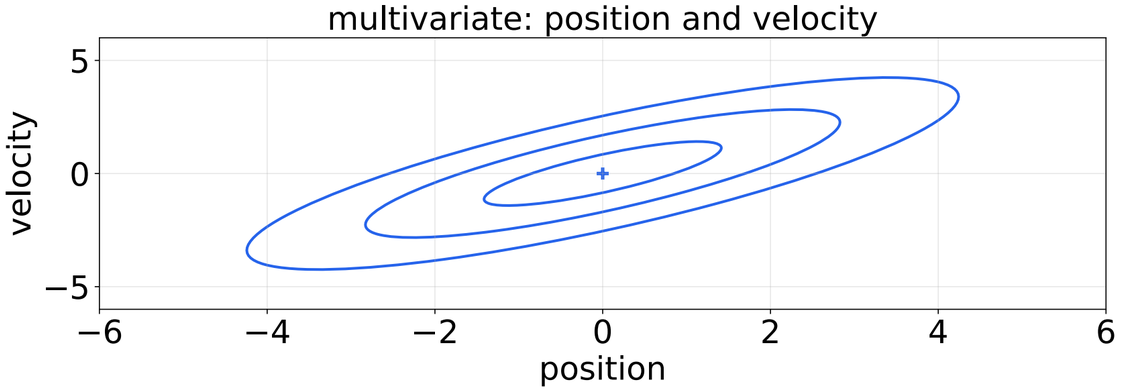
- Introduction
- Multivariate Normal Distributions
- Correlation and Covariance
- Multivariate Normal Distribution Equation
- Using Correlations to Improve Estimates
- Multiplying Multidimensional Gaussians
- Hidden Variables
- Higher Dimensions
- Summary

Introduction

- **1 차원 필터의 한계**: 변수 하나만 표현, 들판을 걷는 개의 위치와 속도 같은 다차원 데이터 표현 불가
- **관계**: 위치와 속도는 서로 관련된 값, 정보는 절대 버리지 않는 것이 원칙
- **목표**: 변수 사이의 관계를 확률로 기술하는 방법
- **효과**: 이 관계를 이용해 필터 성능을 크게 향상

> **핵심**: 위치와 속도처럼 서로 연관된 변수의 관계를 Gaussian 하나로 함께 표현



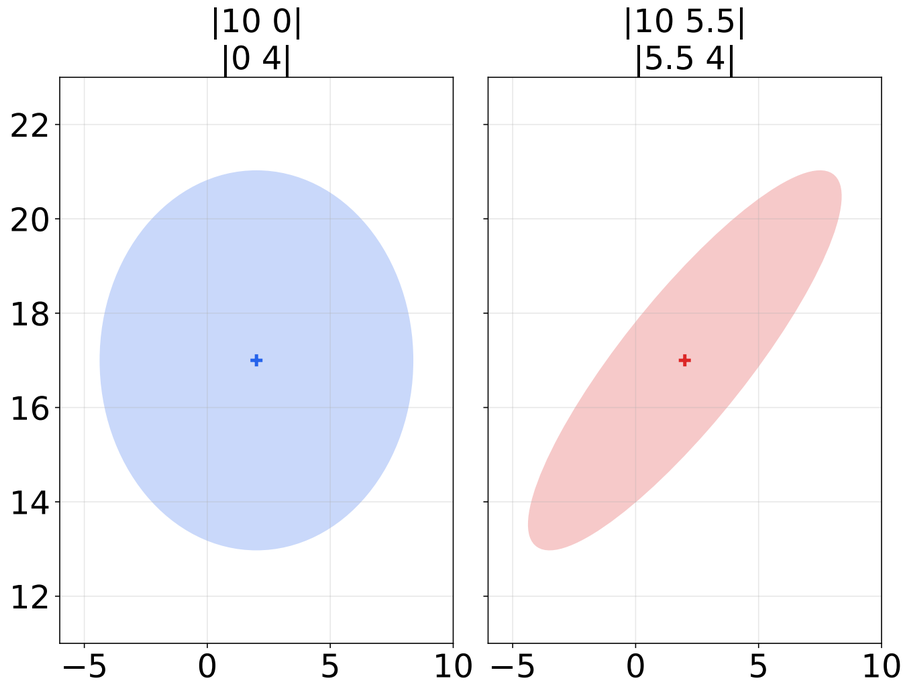


Multivariate Normal Distributions

- **univariate normal**: 변수 하나의 Gaussian, $N(\mu, \sigma^2)$
- **multivariate normal**: 여러 변수의 Gaussian, 예: 항공기 위치, 속도, 가속도 9 차원
- **평균**: N 차원이면 평균 N 개를 열 벡터로 배치

$$\mu = \begin{bmatrix} \mu_1 \\ \mu_2 \\ \vdots \\ \mu_n \end{bmatrix} \qquad \mu = \begin{bmatrix} 2 \\ 17 \end{bmatrix} \qquad \sigma^2 = \begin{bmatrix} 10 \\ 4 \end{bmatrix}$$

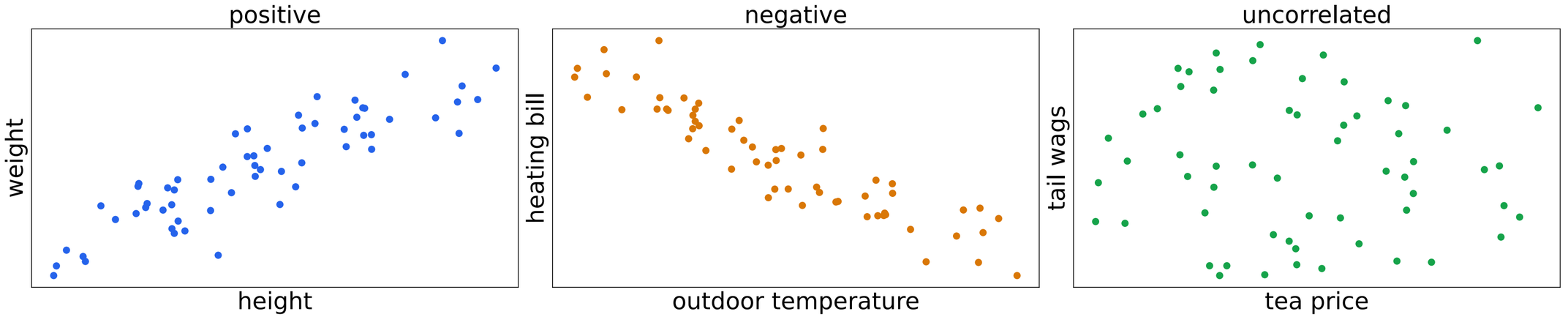
- **분산만으로는 불완전**: 분산 10, 4 가 같아도 오른쪽처럼 모양이 다름
- **covariance 필요**: 키와 몸무게처럼 변수 사이의 상관 정도 표현



Correlation and Covariance

- **covariance**: 두 변수가 함께 변하는 정도, correlated variances의 줄임
- **양의 상관**: 키가 클수록 몸무게도 큼
- **음의 상관**: 바깥 기온이 내려가면 난방비가 올라감
- **무상관, 독립**: 차 가격과 개의 꼬리 흔드는 횟수

> **핵심**: 상관은 예측을 가능하게 함, 키가 훨씬 크면 몸무게도 더 나갈 것으로 예측 가능

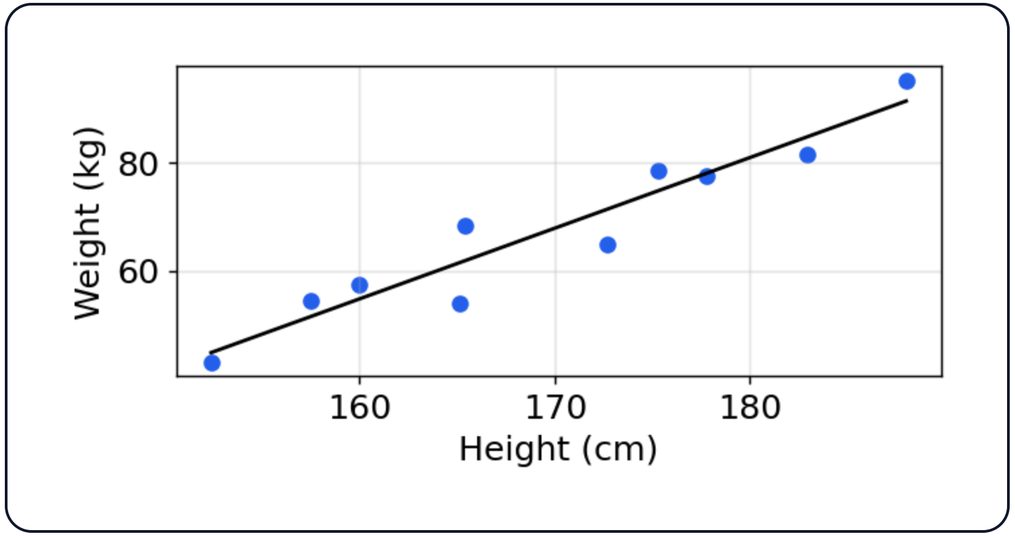


Height and Weight

- 육상부 학생 10 명의 키와 몸무게를 산점도로 그리고 직선을 맞춤
- 점들이 오른쪽 위로 올라가는 양의 상관, 키 173 cm 이면 대략 71 kg 으로 예측 가능
- 상관이 완벽하지 않으므로 예측도 완벽하지 않음, 여기서는 선형 상관만 고려
- **조건**: 키 152 ~ 188 cm, 몸무게 43 ~ 95 kg, 학생 10 명

```
height = [152.4, 157.5, 160.0, 165.1, 165.4,
          172.7, 175.3, 177.8, 182.9, 188.0]  # cm
weight = [43.1, 54.4, 57.6, 54.0, 68.5,
          64.9, 78.5, 77.6, 81.6, 95.3]  # kg

plots.plot_correlated_data(height, weight,
                           'Height (cm)', 'Weight (kg)', False)
```



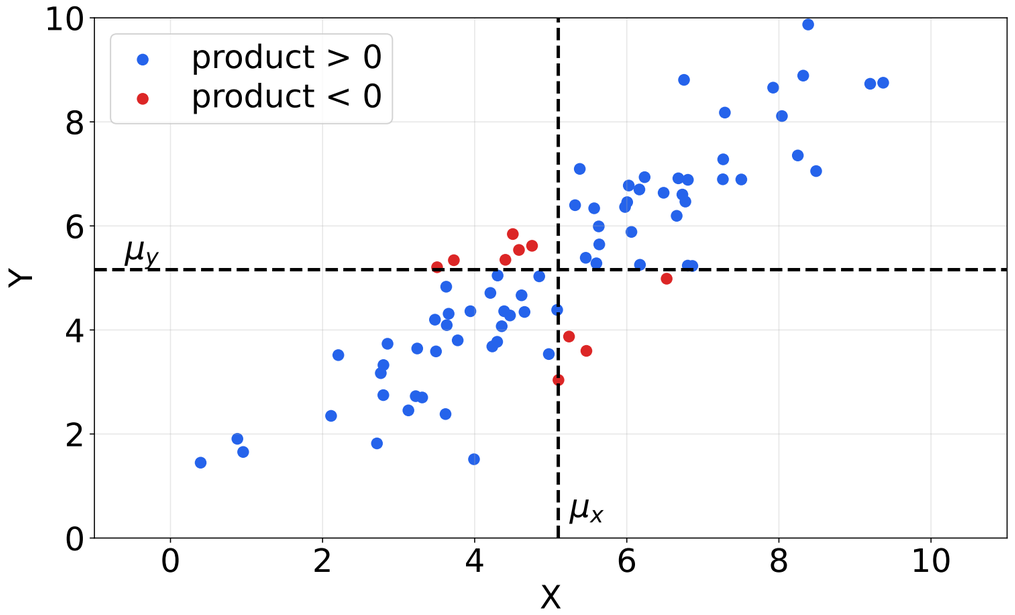
Covariance

$$COV(X,Y) = \sigma_{xy} = E\big[(X-\mu_x)(Y-\mu_y)\big]$$

$$E[X] = \sum_{i=1}^{n} p_i x_i \qquad E[X] = \frac{1}{N}\sum_{i=1}^{n} x_i$$

$$VAR(X) = \sigma_x^2 = E\big[(X-\mu)^2\big]$$

- **기댓값**: 각 데이터가 같은 확률 1/N 이라 가정한 이산 평균
- **분산과 비교**: COV(X, X) 를 계산하면 VAR(X), 분산은 자기 자신과의 공분산
- **부호**: 두 편차의 곱이 대부분 양수면 양의 공분산, 대부분 음수면 음의 공분산



> **핵심**: 분산은 한 변수가 스스로 퍼진 정도, 공분산은 두 변수가 서로에 대해 함께 변하는 정도

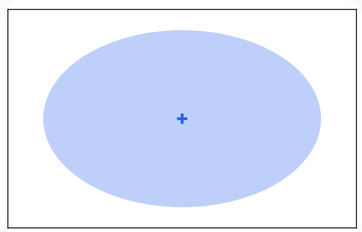
Covariance Matrix

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \sigma_{12} & \cdots & \sigma_{1n} \\ \sigma_{21} & \sigma_2^2 & \cdots & \sigma_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ \sigma_{n1} & \sigma_{n2} & \cdots & \sigma_n^2 \end{bmatrix}$$

- **대각 성분**: 각 변수의 분산, 예: σ_3^2 는 세 번째 변수의 분산
- **비대각 성분**: i 번째와 j 번째 변수의 공분산, 예: σ_{13}
- **대칭**: x 와 y 의 공분산은 y 와 x 의 공분산과 같음, $\sigma_{xy} = \sigma_{yx}$

No correlation

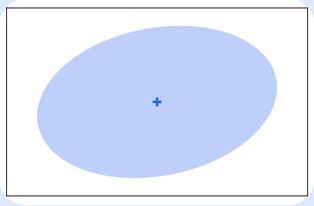
$$\Sigma = \begin{bmatrix} 10 & 0 \\ 0 & 4 \end{bmatrix}$$



- 공분산 0, 선형 상관 없음

Small positive

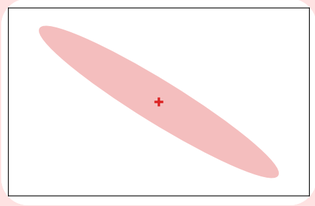
$$\Sigma = \begin{bmatrix} 10 & 1.2 \\ 1.2 & 4 \end{bmatrix}$$



- 분산 10 에 비해 작은 1.2

Large negative

$$\Sigma = \begin{bmatrix} 10 & -6 \\ -6 & 4 \end{bmatrix}$$



- 분산에 견줄 만큼 큰 음의 공분산

Example: Heights and Weights

- 학생 5 명의 키 H (m) 와 몸무게 W (kg) 로 분산과 공분산을 손으로 계산한 뒤 np.cov 와 비교
- np.cov 는 표본 보정으로 1/(N-1) 을 사용해 값이 다르고, bias=1 이면 1/N 으로 손 계산과 일치
- 모집단 전체를 계산할 때는 bias=1, 큰 집단에서 뽑은 표본일 때는 기본값 사용
- **조건**: H = [1.8, 2.0, 1.7, 1.9, 1.6] m, W = [70.1, 91.2, 59.5, 93.2, 53.5] kg

$$\sigma_H^2 = 0.02\ m^2 \qquad \mu_W = 73.5\ kg \qquad \sigma_W^2 = 261.8\ kg^2 \qquad \sigma_{H,W} = \frac{1}{N}\sum_{i=1}^{n}(H_i - \mu_H)(W_i - \mu_W) = 2.18$$

```python
import numpy as np
W = [70.1, 91.2, 59.5, 93.2, 53.5]
H = [1.8, 2.0, 1.7, 1.9, 1.6]
print(np.cov(H, W))
print(np.cov(H, W, bias=1))
```

```
[[  0.025   2.727]
 [  2.727 327.235]]
[[  0.02    2.182]
 [  2.182 261.788]]
```

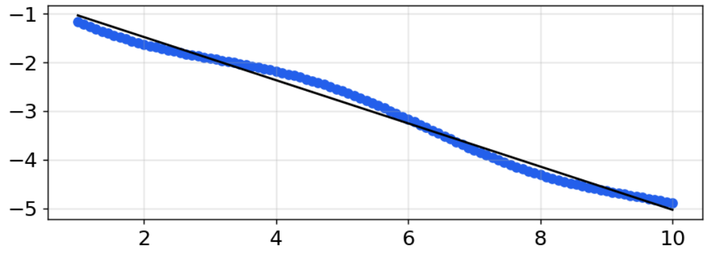
Perfect and Negative Correlation

- 직선 위에 놓인 완전 상관 데이터의 공분산을 출력한 뒤, Y 를 음수로 바꾸고 sin 잡음을 더해 다시 계산
- 완전 상관이면 공분산이 x, y 의 분산과 같음, 두 번째 데이터는 σxy = -3.08 로 음의 상관
- 공분산이 분산 크기에 비해 0 에 가깝지 않아 여전히 강한 상관, 그래프도 거의 직선
- **조건**: X = 1 ~ 10, Y = 1 ~ 10 (완전 상관), 이후 Y = -(1 ~ 5 + 0.2 sin X), 점 100 개

```
X = np.linspace(1, 10, 100)
Y = np.linspace(1, 10, 100)
print(np.cov(X, Y))

X = np.linspace(1, 10, 100)
Y = -(np.linspace(1, 5, 100) + np.sin(X)*.2)
plots.plot_correlated_data(X, Y, equal=False)
print(np.cov(X, Y))
```

```
[[6.956 6.956]
 [6.956 6.956]]
[[ 6.956 -3.084]
 [-3.084  1.387]]
```

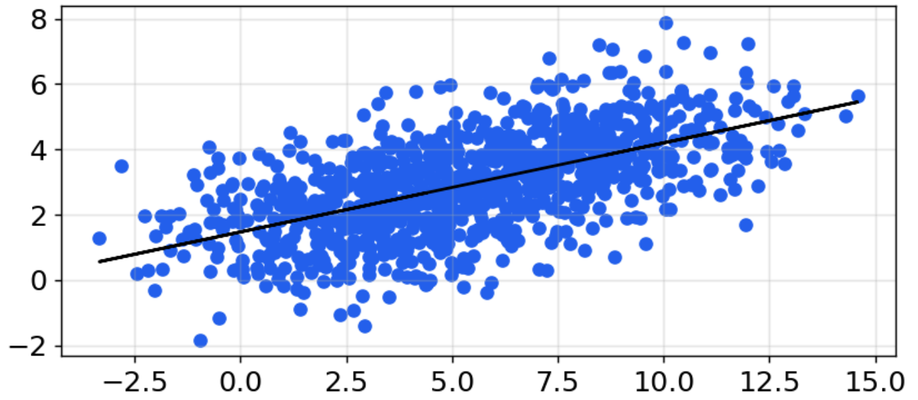


Noise Added to a Line

- 직선 위의 점에 X 는 표준편차 2, Y 는 표준편차 1 의 잡음을 더한 데이터
- 분산에 비해 공분산이 작아져 상관이 약해짐, 직선은 맞출 수 있지만 흩어짐이 훨씬 큼
- **조건**: X = 1 ~ 10 + 2 randn, Y = 1 ~ 5 + randn, 점 1000 개

```
from numpy.random import randn
X = np.linspace(1, 10, 1000) + randn(1000)*2
Y = np.linspace(1, 5, 1000) + randn(1000)
plots.plot_correlated_data(X, Y, equal=False)
print(np.cov(X, Y))
```

```
[[10.895  2.971]
 [ 2.971  2.296]]
```

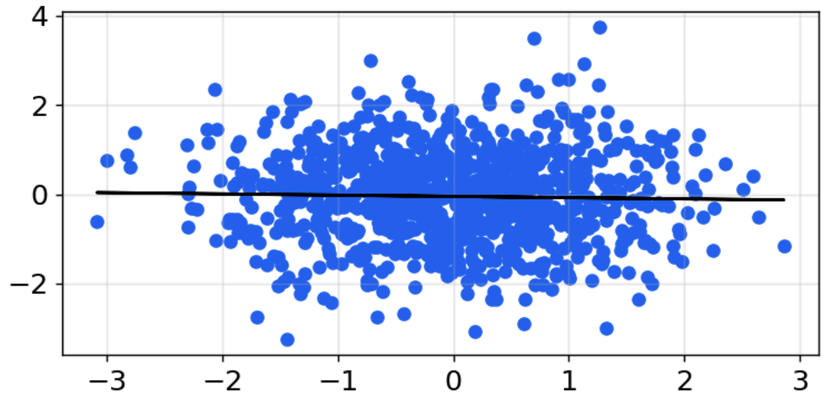


Completely Random Data

- 서로 무관한 두 난수 X, Y 의 공분산 행렬 계산
- 공분산이 0 에 매우 가까움, 그래프에서도 맞출 직선이 없음, 수직선도 수평선만큼 설득력 없음
- **조건**: X, Y 각각 표준 정규 난수 1000 개

```
1  X = randn(1000)
2  Y = randn(1000)
3  plots.plot_correlated_data(X, Y, equal=False)
4  print(np.cov(X, Y))
```

```
[[ 0.927 -0.026]
 [-0.026  1.028]]
```



Multivariate Normal Distribution Equation

$$f(x, \mu, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{1}{2}(x-\mu)^2/\sigma^2\right] \qquad f(\mathbf{x}, \mu, \Sigma) = \frac{1}{\sqrt{(2\pi)^n|\Sigma|}} \exp\left[-\frac{1}{2}(\mathbf{x}-\mu)^T\Sigma^{-1}(\mathbf{x}-\mu)\right]$$

	Univariate	Multivariate	의미
상태	x	$\mathbf{x}$	n 개 변수의 열 벡터
평균	μ	μ	각 변수의 평균을 모은 벡터
불확실성	σ^2	Σ	분산과 공분산을 모은 공분산 행렬
나눗셈	$1/\sigma^2$	Σ^{-1}	역행렬, 스칼라의 역수 역할
정규화	$\sqrt{2\pi\sigma^2}$	$\sqrt{(2\pi)^n\|\Sigma\|}$	행렬식으로 전체 부피가 1 이 되도록

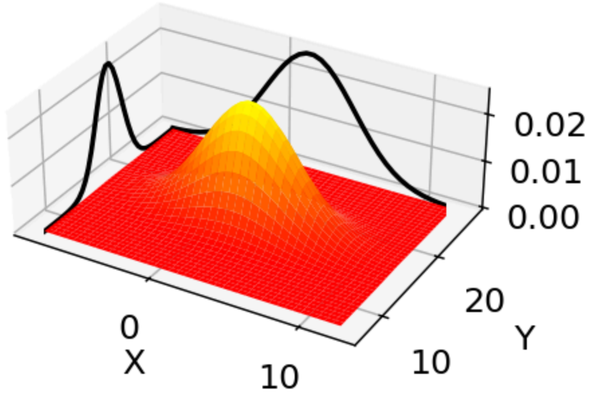
핵심: 다변량 식은 단변량 식의 스칼라를 벡터와 행렬로 바꾼 것, 계산은 라이브러리 함수로 처리

3D Plot of a Multivariate Gaussian

- 평균 [2, 17], 공분산 diag(10, 4) 인 2 차원 Gaussian 의 확률 밀도를 3D 곡면으로 표시
- 벽면의 곡선은 x, y 의 분산을 투영한 Gaussian, x 쪽이 더 넓고 곡면의 꼭대기는 평균 위치
- 개의 위치라면 (2, 17) 근처가 가장 유력, (5, 14) 는 중간, (10, 10) 은 매우 낮은 밀도
- **조건**: mean = [2, 17], cov = [[10, 0], [0, 4]], 밀도이므로 확률은 곡면 아래 부피로 계산

```python
mean = [2., 17.]
cov = [[10., 0.],
       [0., 4.]]

plots.plot_3d_covariance(mean, cov)
```



Computing the Density

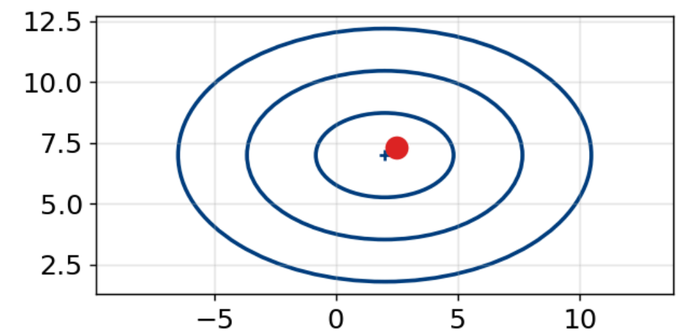
- (2, 7) 에 있다고 믿는 개가 (2.5, 7.3) 에 있을 확률 밀도를 두 라이브러리 함수로 계산
- 두 함수 모두 0.0315 로 같은 값, 그래프의 점은 1σ 타원 안쪽으로 평균에 가까운 위치
- 개는 x, y 방향으로 서로 독립적으로 움직일 수 있으므로 상관 없음, 공분산 행렬 이름은 P
- **조건**: x = [2.5, 7.3], mu = [2, 7], x 분산 8, y 분산 3, 공분산 0

```python
from filterpy.stats import multivariate_gaussian
import scipy.stats

x = np.array([2.5, 7.3])
mu = np.array([2.0, 7.0])
P = [[8., 0.],
     [0., 3.]]
print('{:.4f}'.format(multivariate_gaussian(x, mu, P)))
print('{:.4f}'.format(scipy.stats.multivariate_normal(mu, P).pdf(x)))

plot_covariance_ellipse(mu, P, std=[1, 2, 3])
plt.scatter(*x, s=120, color='C3', zorder=3);
```

```
0.0315
0.0315
```

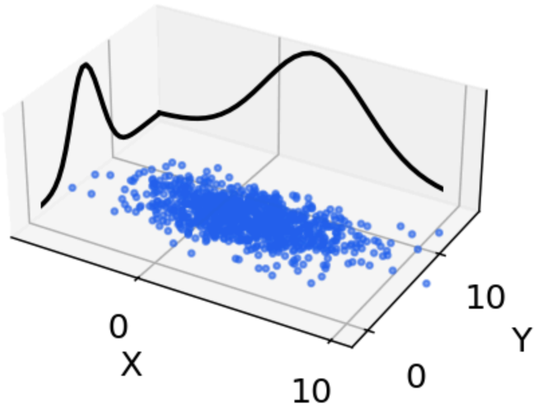


Joint and Marginal Probability

- joint probability P(x, y): x 와 y 가 함께 일어날 확률, 주사위 두 개면 P(2, 5) = 1/6 × 1/6 = 1/36
- marginal probability: 다른 변수를 무시한 한 변수의 확률, 벽면 곡선이 x, y 의 marginal
- 분포를 따르는 점 1000 개를 개의 가능한 위치로 생각, 점이 많은 (2, 7) 이 (-5, 5) 보다 유력
- **조건**: mu = [2, 7], P = [[8, 0], [0, 3]], 표본 1000 개, 다변량 Gaussian 의 marginal 도 Gaussian

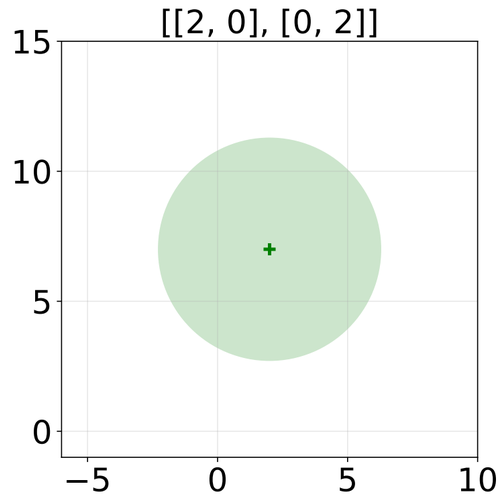
```
mu = [2.0, 7.0]
P = [[8., 0.],
     [0., 3.]]

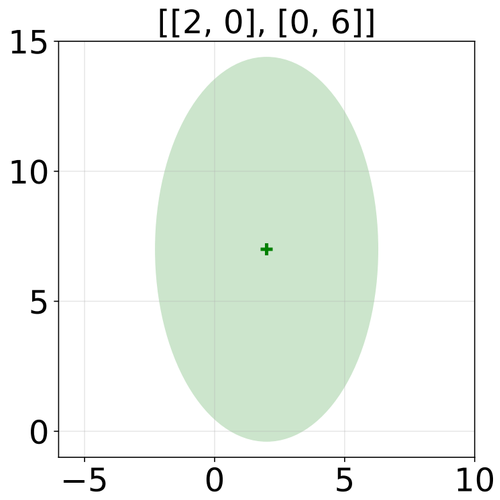
plots.plot_3d_sampled_covariance(mu, P)
```

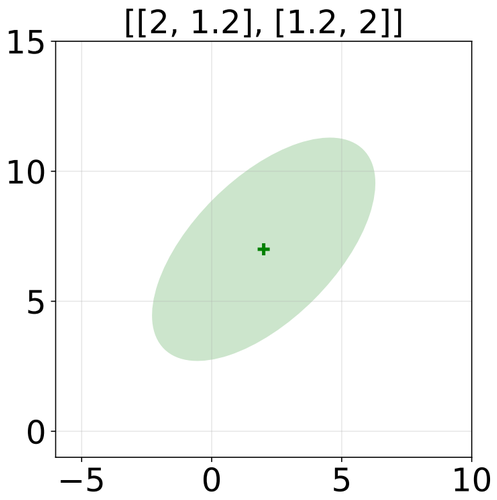


Error Ellipses

- **3D 그림의 한계**: 분산 비교와 상관 정도를 읽기 어려움
- **등고선**: 3D 곡면의 수평 단면, 여기서는 3σ 단면
- **이름**: 단면은 항상 타원, error ellipse, confidence ellipse
- **해석**: 크기는 믿음의 오차, 모양은 x, y 오차의 기하 관계







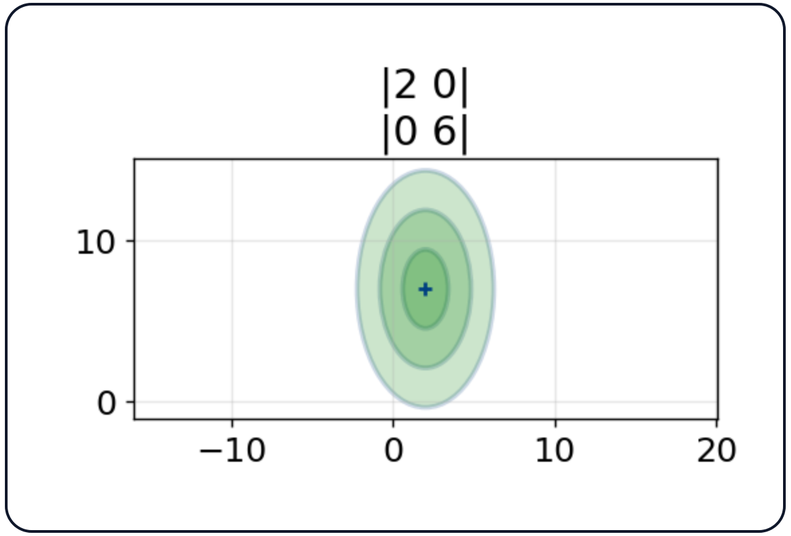
- **원**: x, y 오차가 같은 정도로 가능
- **세로 타원**: y 가 x 보다 더 불확실
- **기운 타원**: 비대각 성분이 0 이 아님, 상관

Plotting Standard Deviations

- 공분산 [[2, 0], [0, 6]] 의 1σ, 2σ, 3σ 타원을 한 번에 그림, 기본값은 1σ
- std=3 또는 variance=3**2 는 3σ, std=[1, 2, 3] 은 세 개 모두 표시
- 단색으로 칠해져 있어도 두 표준편차 사이에서 확률 밀도가 일정하다는 뜻은 아님
- **조건**: 평균 (2, 7), P = [[2, 0], [0, 6]], std = [1, 2, 3]

```python
from filterpy.stats import plot_covariance_ellipse
import matplotlib.pyplot as plt

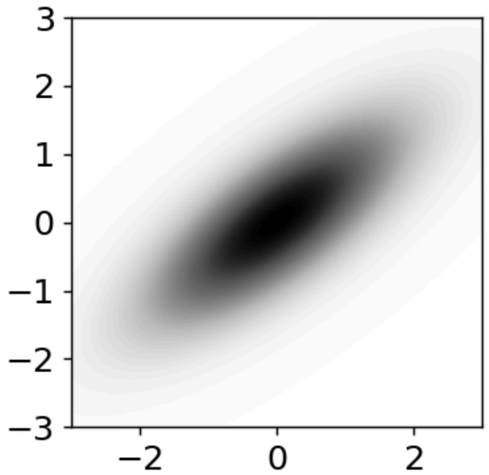
P = [[2, 0], [0, 6]]
plot_covariance_ellipse((2, 7), P, fc='g', alpha=0.2,
                        std=[1, 2, 3],
                        title='|2 0|\n|0 6|')
```



Probability Density Shading

- 공분산 [[2, 1.2], [1.2, 1.3]] 의 확률 밀도를 회색 농도로 표시, 진할수록 밀도가 높음
- 중심에서 바깥으로 연속적으로 옅어짐, 타원 경계는 특정 밀도의 등고선일 뿐임
- **조건**: cov = [[2, 1.2], [1.2, 1.3]], 평균 (0, 0), 범위 -3 ~ 3

```
plots.plot_cov_ellipse_colormap(
    cov=[[2, 1.2], [1.2, 1.3]]);
```



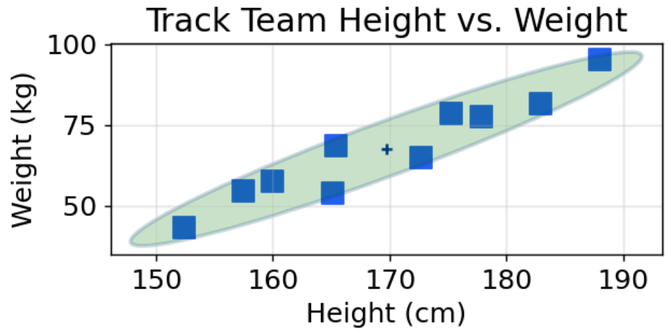
Height and Weight Ellipse

- np.cov 로 키와 몸무게의 공분산을 구하고 데이터 위에 2σ 공분산 타원을 겹쳐 그림
- 좁은 타원은 강한 상관, 주어진 키에 대해 가능한 몸무게 범위가 좁음
- 타원이 오른쪽으로 기울어 양의 상관, 데이터가 모집단 전체이므로 bias=1
- **조건**: 육상부 10 명, 키 (cm), 몸무게 (kg), std = 2

```python
height = [152.4, 157.5, 160.0, 165.1, 165.4,
          172.7, 175.3, 177.8, 182.9, 188.0]  # cm
weight = [43.1, 54.4, 57.6, 54.0, 68.5,
          64.9, 78.5, 77.6, 81.6, 95.3]  # kg
cov_hw = np.cov(np.vstack((height, weight)), bias=1)
print(cov_hw)

plt.scatter(height, weight, s=120, marker='s')
plt.title('Track Team Height vs. Weight')
plt.xlabel('Height (cm)'); plt.ylabel('Weight (kg)')
plot_covariance_ellipse((np.mean(height), np.mean(weight)),
                        cov_hw, fc='g', alpha=0.2,
                        axis_equal=False, std=2)
```

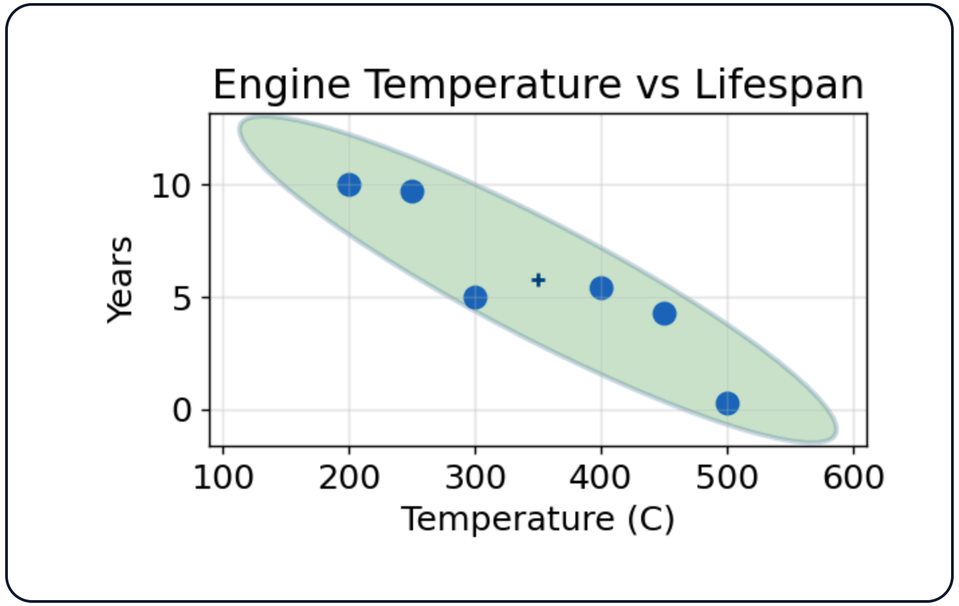
```
[[119.697 156.624]
 [156.624 224.962]]
```



Engine Temperature vs Lifespan

- 엔진 최고 온도와 수명 데이터에 2σ 공분산 타원을 겹쳐 그림
- 타원이 왼쪽으로 기울어 음의 상관, 온도가 높을수록 수명이 짧아짐
- **조건**: 최고 온도 200 ~ 500 C, 수명 0.3 ~ 10 년, 데이터 6 개, std = 2

```
max_temp = [200, 250, 300, 400, 450, 500]
lifespan = [10, 9.7, 5, 5.4, 4.3, 0.3]
plt.scatter(max_temp, lifespan, s=80)
cov = np.cov(np.vstack((max_temp, lifespan)))
plot_covariance_ellipse((np.mean(max_temp), np.mean(lifespan)),
                        cov, fc='g', alpha=0.2,
                        axis_equal=False, std=2)
plt.title('Engine Temperature vs Lifespan')
plt.xlabel('Temperature (C)'); plt.ylabel('Years');
```



Interactive Covariance

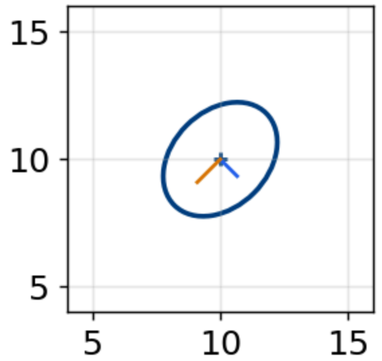
- x 분산, y 분산, 공분산을 슬라이더로 바꾸며 공분산 타원과 반축(semi-axis)의 변화 관찰
- 공분산이 커질수록 타원이 기울고 가늘어짐, 분산은 타원의 x, y 폭을 결정
- **조건**: 평균 (10, 10), var_x, var_y 0 ~ 20, cov_xy 0 ~ 50

```python
from ipywidgets import interact, FloatSlider

def plot_covariance(var_x, var_y, cov_xy):
    P1 = [[var_x, cov_xy], [cov_xy, var_y]]
    plot_covariance_ellipse((10, 10), P1, axis_equal=False,
                            show_semiaxis=True)
    plt.xlim(4, 16)
    plt.gca().set_aspect('equal')
    plt.ylim(4, 16)

interact(plot_covariance,
         var_x=FloatSlider(5, min=0, max=20),
         var_y=FloatSlider(5, min=0, max=20),
         cov_xy=FloatSlider(1.5, min=0, max=50, step=.2));
```

var_x 5 var_y 5
cov_xy 1.5

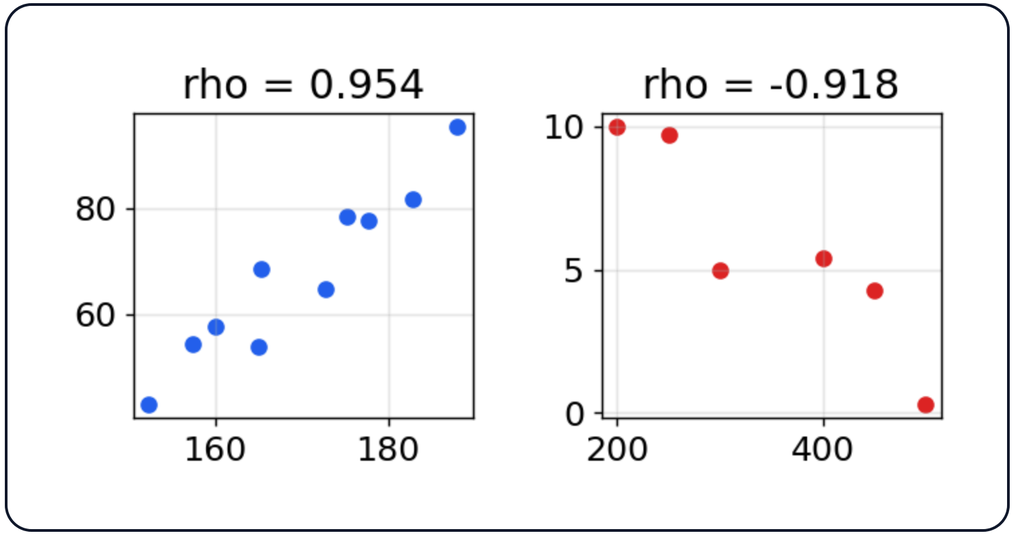


Pearson's Correlation Coefficient

- 키와 몸무게, 엔진 온도와 수명의 Pearson 상관계수를 scipy.stats.pearsonr 로 계산해 제목에 표시
- 키와 몸무게는 0.954 로 강한 양의 상관, 온도와 수명은 -0.918 로 강한 음의 상관
- 공분산은 단위가 있지만 (예: m^2) 상관계수는 단위 없는 -1 ~ 1 의 비율
- **조건**: 육상부 10 명 (cm, kg), 엔진 데이터 6 개, $\rho_{xy} = COV(X,Y)/(\sigma_x \sigma_y)$, 범위 -1 ~ 1

```python
height = [152.4, 157.5, 160.0, 165.1, 165.4,
          172.7, 175.3, 177.8, 182.9, 188.0]  # cm
weight = [43.1, 54.4, 57.6, 54.0, 68.5,
          64.9, 78.5, 77.6, 81.6, 95.3]  # kg
max_temp = [200, 250, 300, 400, 450, 500]
lifespan = [10, 9.7, 5, 5.4, 4.3, 0.3]
from scipy.stats import pearsonr

r1 = pearsonr(height, weight)[0]
r2 = pearsonr(max_temp, lifespan)[0]
plt.subplot(121); plt.scatter(height, weight)
plt.title('rho = {:.3f}'.format(r1))
plt.subplot(122); plt.scatter(max_temp, lifespan, color='C3')
plt.title('rho = {:.3f}'.format(r2));
```



Correlation and Ellipse Shape

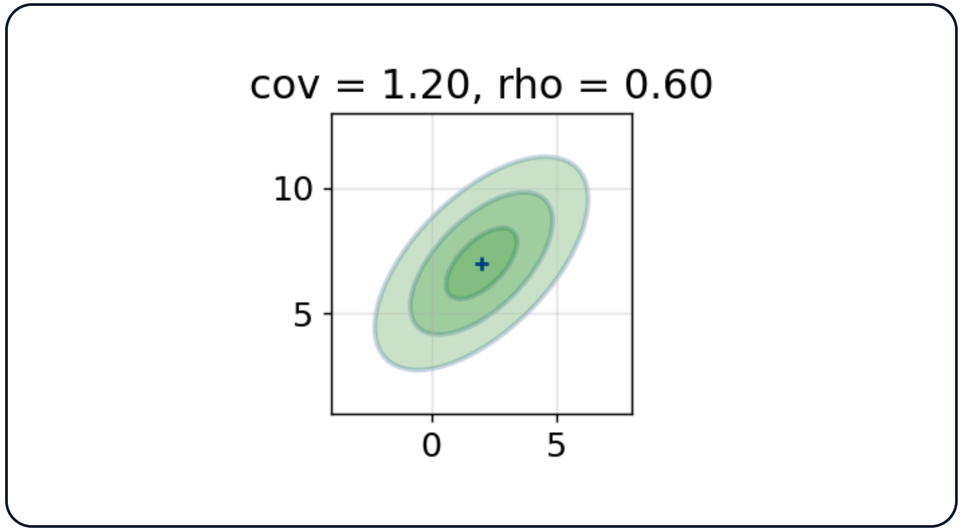
- 두 분산을 2 로 고정하고 상관계수 rho 만 바꾸며 공분산 타원의 모양 변화 관찰
- rho = 0 이면 원, 1 이나 -1 에 가까울수록 오른쪽 또는 왼쪽으로 기운 가는 타원
- **조건**: 평균 (2, 7), 분산 2, cov = rho σx σy, rho -0.95 ~ 0.95, std = [1, 2, 3]

```python
from ipywidgets import interact, FloatSlider

def plot_rho(rho):
    var = 2.
    cov_xy = rho * np.sqrt(var) * np.sqrt(var)
    P = [[var, cov_xy], [cov_xy, var]]
    plot_covariance_ellipse((2, 7), P, fc='g', alpha=0.2,
                            std=[1, 2, 3], axis_equal=False)
    plt.xlim(-4, 8); plt.ylim(1, 13)
    plt.gca().set_aspect('equal')
    plt.title('cov = {:.2f}, rho = {:.2f}'.format(cov_xy, rho))

interact(plot_rho,
         rho=FloatSlider(0.6, min=-0.95, max=0.95, step=0.05));
```

rho 0.6

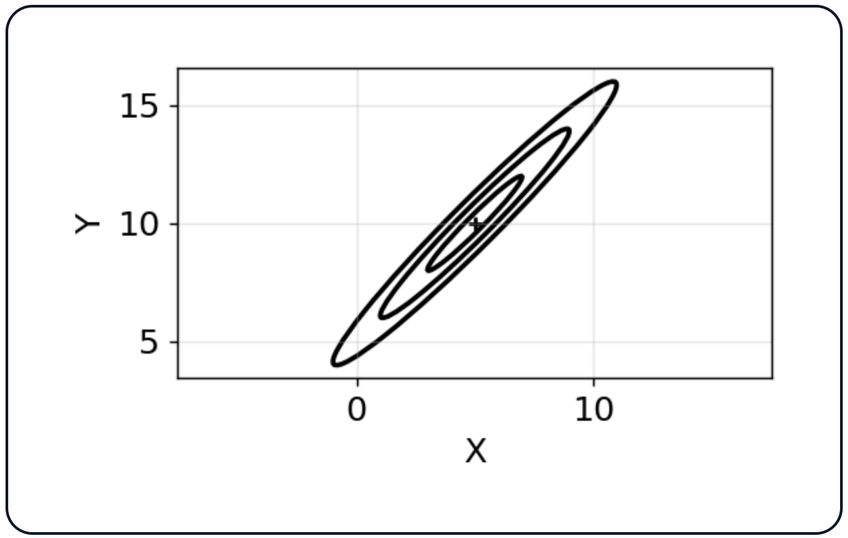


Strongly Correlated Position

- 개가 (5, 10) 에 있다고 믿고 x, y 표준편차는 각각 2 m 이지만 강하게 상관된 경우의 공분산 타원
- 1σ, 2σ, 3σ 타원 모두 대각선 방향으로 매우 가늘게 놓임
- x 를 알면 상관을 통해 y 를 좁은 범위로 추론 가능
- **조건**: 평균 (5, 10), P = [[4, 3.9], [3.9, 4]], std = [1, 2, 3]

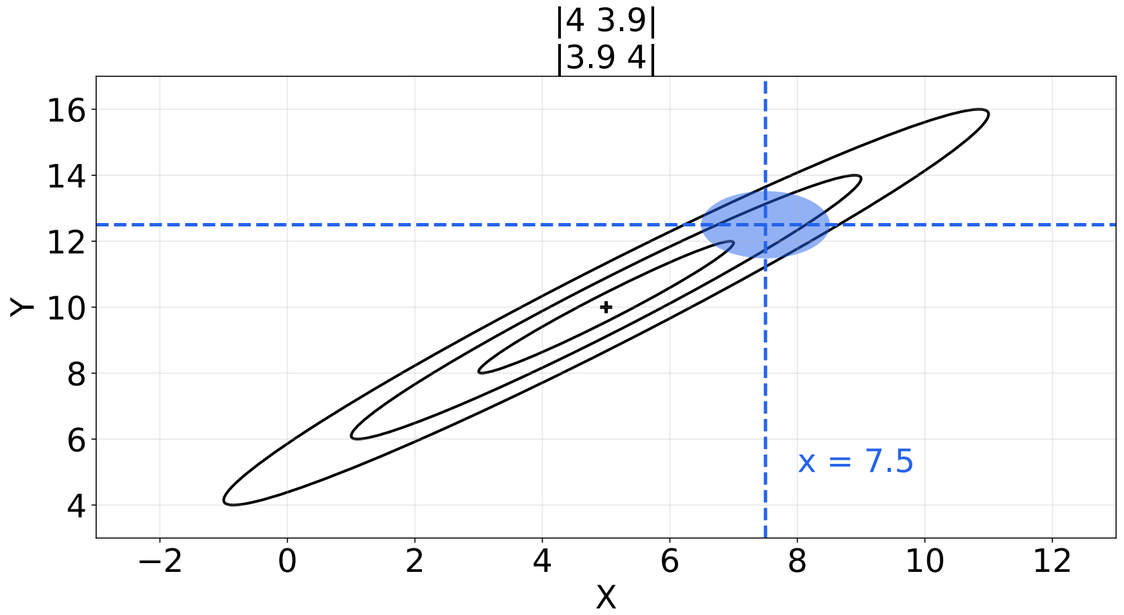
```
P = [[4, 3.9], [3.9, 4]]

plot_covariance_ellipse((5, 10), P, ec='k', std=[1, 2, 3])
plt.xlabel('X')
plt.ylabel('Y');
```



Knowing x = 7.5

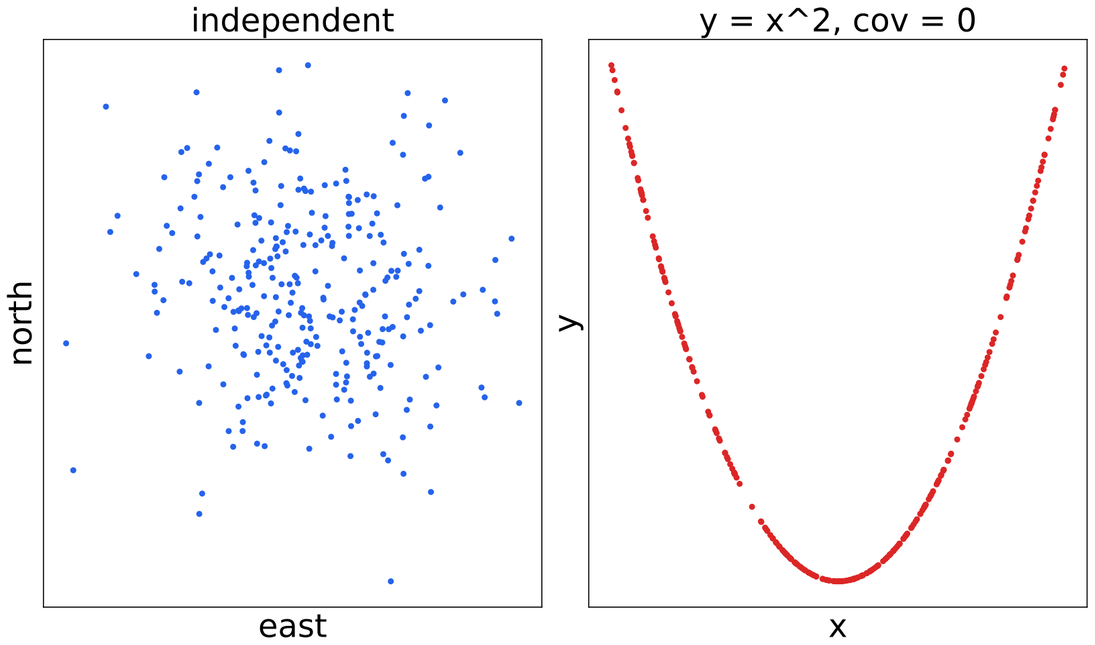
- **질문**: x = 7.5 임을 알게 되면 y 는 어디일까
- **추론**: 위치는 3σ 타원 안에 있을 가능성이 매우 높음, x 와 y 가 상관되어 있으므로 y 범위가 좁혀짐
- **결과**: y 는 12 근처일 가능성이 높고 y = -10 은 거의 불가능
- **파란 원**: y 의 가능한 범위를 나타낸 개념도, 수학적으로 정확한 모양은 아님



핵심: 상관이 있으면 한 변수의 정보로 다른 변수를 좁은 범위로 추정

Correlation and Independence

- **독립**: 각 변수가 따로 변할 수 있음, 들판에서는 동서, 남북, 그 조합으로 자유롭게 이동
- **독립이면 무상관**: 독립 변수는 항상 상관이 없음
- **무상관이어도 종속 가능**: $y = x^2$ 는 y 가 x 에 의해 결정되지만 선형 상관은 0
- **이유**: 상관은 선형 관계만 측정하는 값



> **핵심**: 독립 → 무상관은 성립, 무상관 → 독립은 일반적으로 성립하지 않음

Multiplying Multidimensional Gaussians

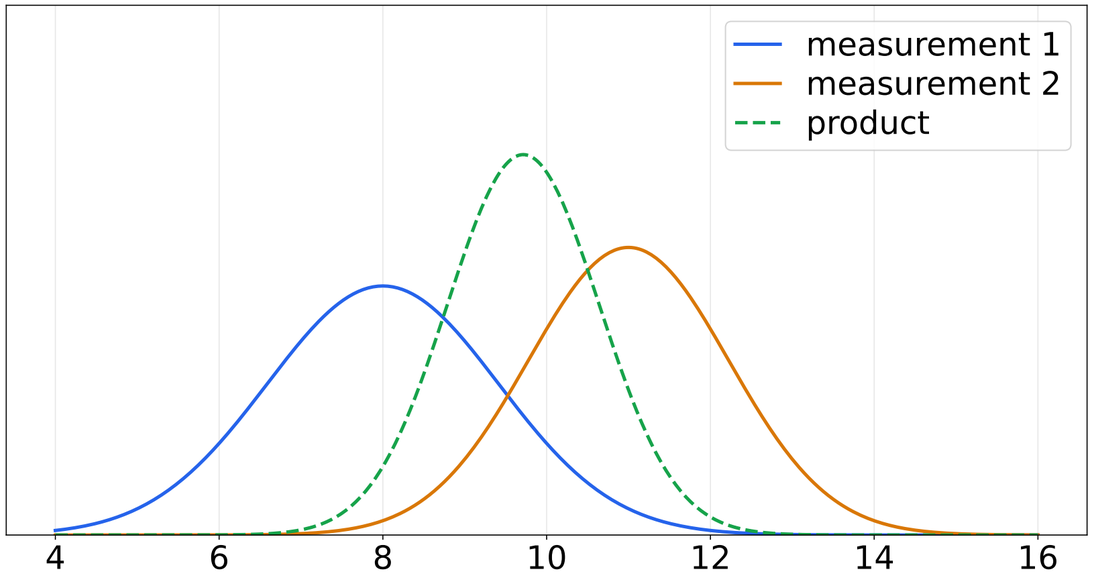
- **1 차원**: 두 정보를 곱하면 분산이 더 작은 Gaussian, 더 확신
- **다차원**: 공분산 행렬 Σ 로 같은 계산, Bayes 정리로 유도

$$\mu = \Sigma_2(\Sigma_1 + \Sigma_2)^{-1}\mu_1 + \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\mu_2$$

$$\Sigma = \Sigma_1(\Sigma_1 + \Sigma_2)^{-1}\Sigma_2$$

$$\mu = \frac{\sigma_1^2\mu_2 + \sigma_2^2\mu_1}{\sigma_1^2 + \sigma_2^2} \qquad \sigma^2 = \frac{\sigma_1^2\sigma_2^2}{\sigma_1^2 + \sigma_2^2}$$

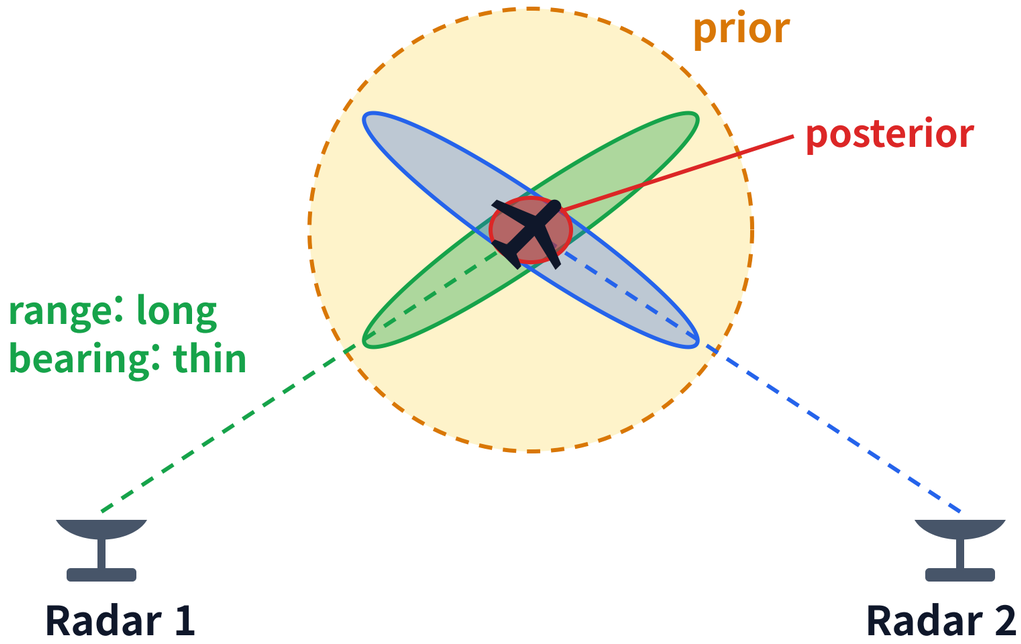
$$\mu \approx \frac{\Sigma_2\mu_1 + \Sigma_1\mu_2}{\Sigma_1 + \Sigma_2} \qquad \Sigma \approx \frac{\Sigma_1\Sigma_2}{\Sigma_1 + \Sigma_2}$$



핵심: 역행렬은 역수와 비슷한 역할 ($AA^{-1} = I$), 역행렬을 나눗셈처럼 쓰면 단변량 식과 같은 형태

Tracking an Aircraft with Radar

- **상황**: 두 레이더로 항공기 추적, 고도는 무시하고 2 차원으로 표현
- **레이더 측정**: 목표까지의 거리(range)와 방위(bearing)
- **prior**: 항공기 위치를 잘 모르는 상태, 노란 원 모양의 큰 공분산
- **측정 오차**: bearing 은 정확, range 는 부정확 → 레이더를 향한 길고 가는 타원
- **posterior**: prior 와 측정(evidence)을 곱해 얻는 평균과 공분산



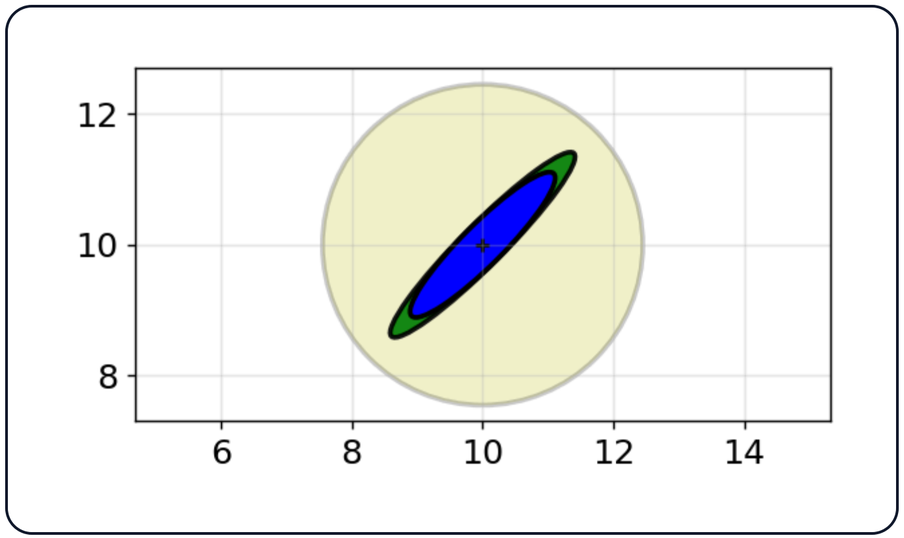
Prior and First Radar

- 노란 prior 에 왼쪽 아래 레이더의 측정(초록)을 곱해 posterior(파랑)를 계산
- posterior 는 측정과 같은 모양과 위치를 유지하면서 더 작아짐, 정보가 늘어 불확실성 감소
- 공분산 모양은 항공기와 레이더의 물리적 배치를 그대로 반영
- **조건**: prior P0 = [[6, 0], [0, 6]], 레이더 1 P1 = [[2, 1.9], [1.9, 2]], 평균 모두 (10, 10)

```python
from filterpy.stats import multivariate_multiply

P0 = [[6, 0], [0, 6]]
P1 = [[2, 1.9], [1.9, 2]]
P2 = multivariate_multiply((10, 10), P0, (10, 10), P1)[1]

plot_covariance_ellipse((10, 10), P0, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P1, ec='k', fc='g', alpha=0.9)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='b')
```

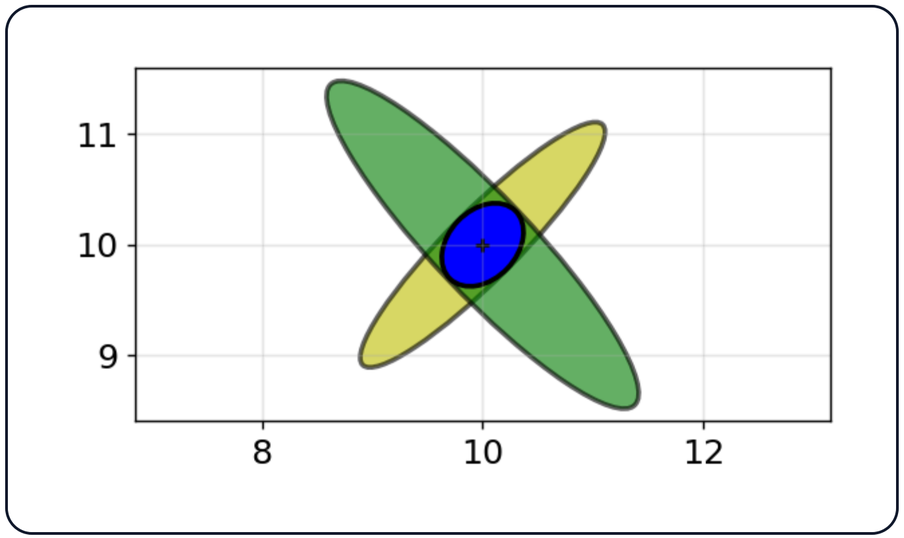


Second Radar

- 직전 posterior 를 새 prior(노랑)로, 오른쪽 아래 두 번째 레이더의 측정(초록)과 곱함
- 항공기가 있을 수 있는 곳은 두 타원이 겹치는 곳뿐, 곱한 결과(파랑)가 매우 작은 Gaussian
- 삼각측량 코드를 따로 작성하지 않아도 Gaussian 곱의 결과로 자연스럽게 삼각측량
- **조건**: prior P2, 레이더 2 P3 = [[2, -1.9], [-1.9, 2.2]], 평균 (10, 10)

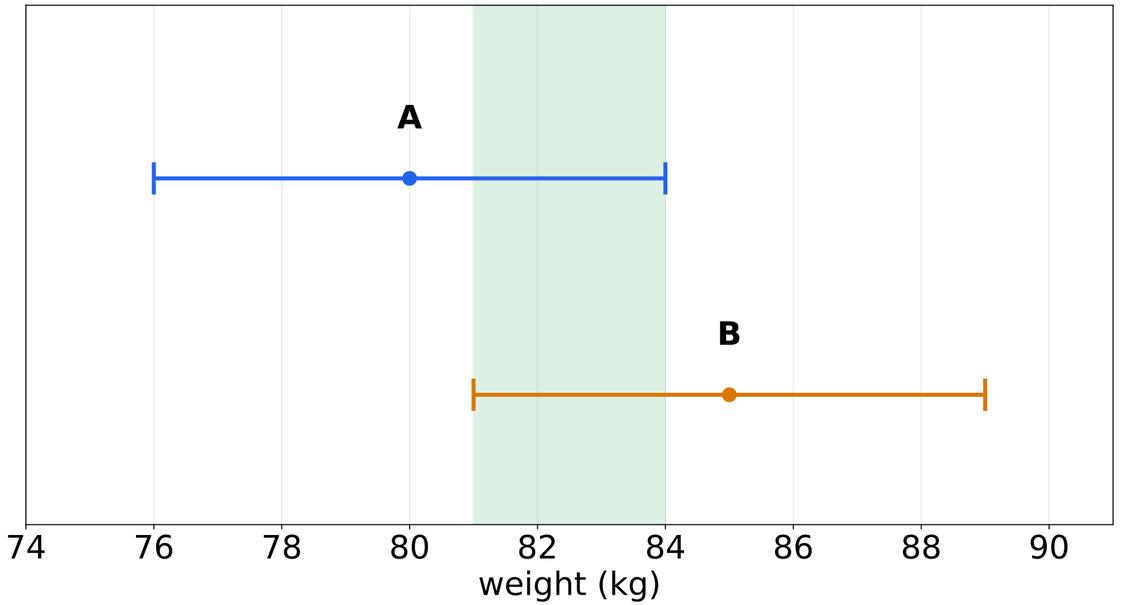
```
P3 = [[2, -1.9], [-1.9, 2.2]]
P4 = multivariate_multiply((10, 10), P2, (10, 10), P3)[1]

plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.6)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P4, ec='k', fc='b')
```



Triangulation

- **1 차원 비교**: 저울 A 80 ± 4 kg, 저울 B 85 ± 4 kg 의 오차 막대
- **겹치는 구간**: 추정은 두 오차 막대가 겹치는 81 ~ 84 kg 안에 있어야 함
- **2 차원**: 두 타원이 겹치는 영역, 곱한 Gaussian 이 그 교집합
- **기하**: 결과 모양은 문제의 기하, 즉 센서 배치를 반영



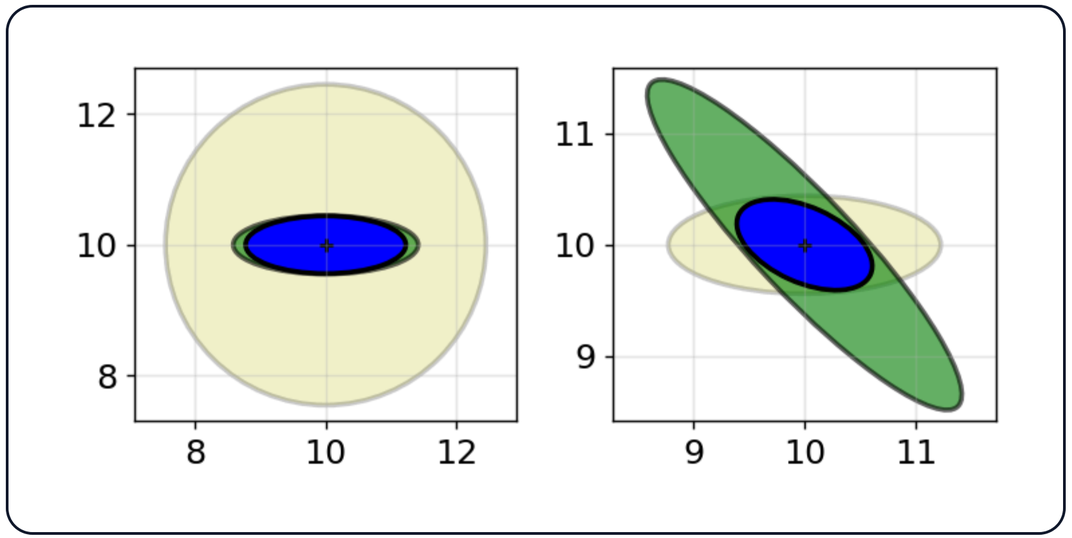
> **핵심**: Gaussian 곱이 곧 삼각측량, 서로 다른 방향의 정보가 만나면 정확한 추정

Radar Directly to the Left

- 첫 레이더가 항공기 바로 왼쪽에 있는 배치, 측정 오차는 x 방향으로 긴 수평 타원
- 왼쪽 그림은 prior 와 첫 측정의 곱, 오른쪽 그림은 같은 위치의 두 번째 레이더 측정을 다시 곱한 결과
- 두 레이더가 항공기 기준으로 직교하지 않아 삼각측량이 최적이 아님, 앞 배치보다 덜 정확
- **조건**: P0 = [[6, 0], [0, 6]], 레이더 1 P1 = [[2, 0], [0, 0.2]], 레이더 2 P3 = [[2, -1.9], [-1.9, 2.2]]

```
P0 = [[6, 0], [0, 6]]
P1 = [[2, 0], [0, .2]]
P2 = multivariate_multiply((10, 10), P0, (10, 10), P1)[1]
plt.subplot(121)
plot_covariance_ellipse((10, 10), P0, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P1, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='b')

P3 = [[2, -1.9], [-1.9, 2.2]]
P4 = multivariate_multiply((10, 10), P2, (10, 10), P3)[1]
plt.subplot(122)
plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10, 10), P3, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse((10, 10), P4, ec='k', fc='b')
```

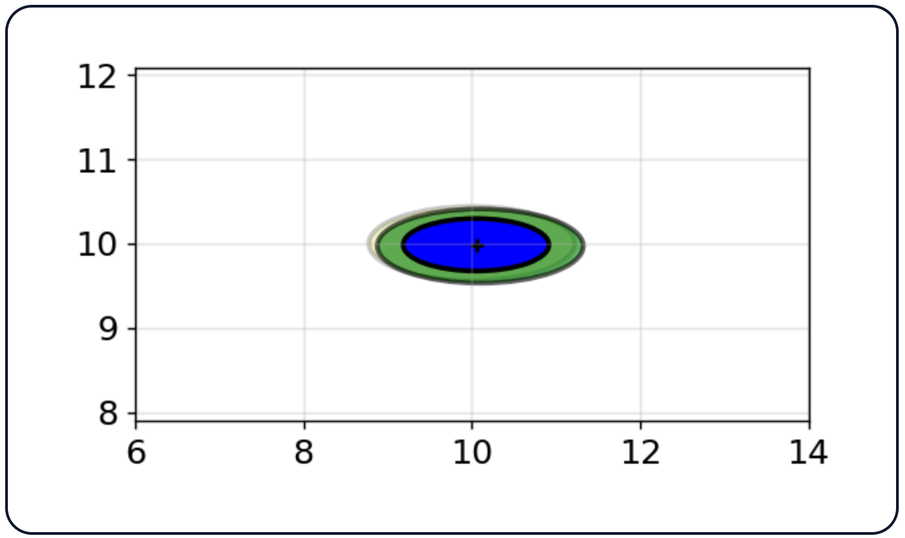


Same Radar Twice

- 같은 레이더로 짧은 시간 간격을 두고 두 번 측정한 상황
- 두 공분산 타원이 거의 겹치므로 곱한 결과(파랑)도 크게 줄지 않아 오차가 여전히 큼
- 같은 방향의 정보만 반복되면 삼각측량 효과가 없음
- **조건**: 두 측정의 공분산 모두 P2, 평균 (10, 10) 과 (10.1, 9.97)

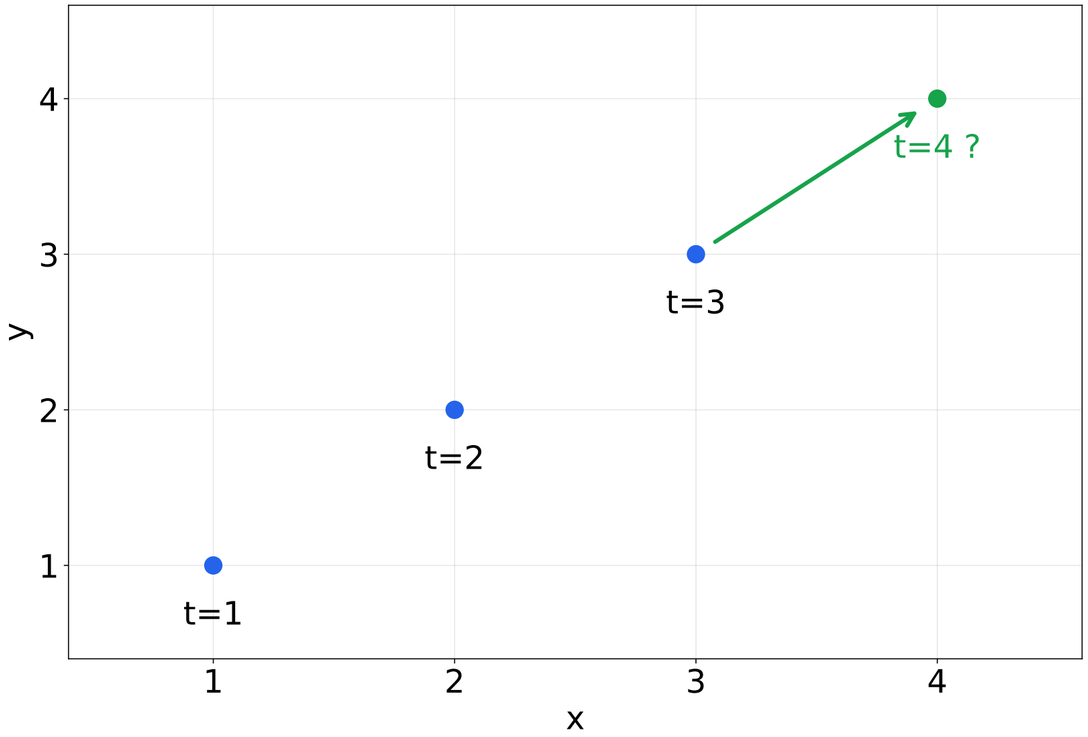
```
P5 = multivariate_multiply((10, 10), P2, (10.1, 9.97), P2)

plot_covariance_ellipse((10, 10), P2, ec='k', fc='y', alpha=0.2)
plot_covariance_ellipse((10.1, 9.97), P2, ec='k', fc='g', alpha=0.6)
plot_covariance_ellipse(P5[0], P5[1], ec='k', fc='b')
plt.xlim(6, 14);
```



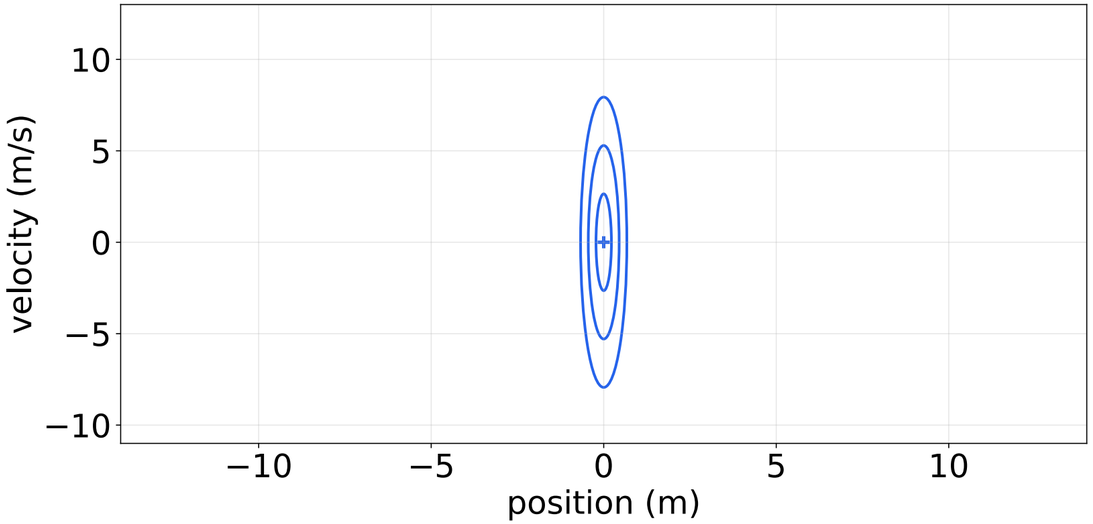
Predicting the Next Position

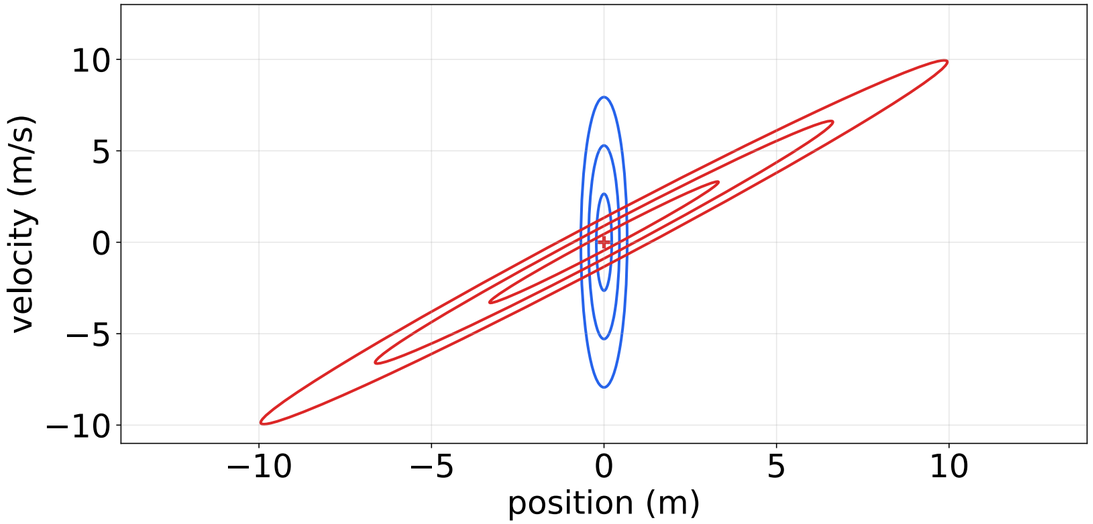
- **데이터**: t = 1, 2, 3 초의 항공기 x, y 좌표
- **직관**: 직선 비행 중이고 항공기는 갑자기 방향을 바꿀 수 없음 → t=4 에 (4, 4)
- **추론 근거**: 일정한 속도, 매 단계 x, y 로 1 씩 이동한다고 추론
- **g-h 필터와 같은 원리**: 체중과 체중 증가율을 함께 추적한 것처럼 위치와 속도를 함께 추적
- **범위**: 4 차원 그림은 그릴 수 없으므로 x 축의 위치와 속도만 표시, 수학은 고차원으로 일반화



Position and Velocity

- **t = 1 의 믿음**: 위치 x = 0 은 꽤 확실, 속도는 전혀 모름 → 좁고 높은 타원
- **위치와 속도의 상관**: 속도 5 m/s 면 1 초 뒤 위치 5 m, -10 m/s 면 -10 m → 대각선 타원(빨강)

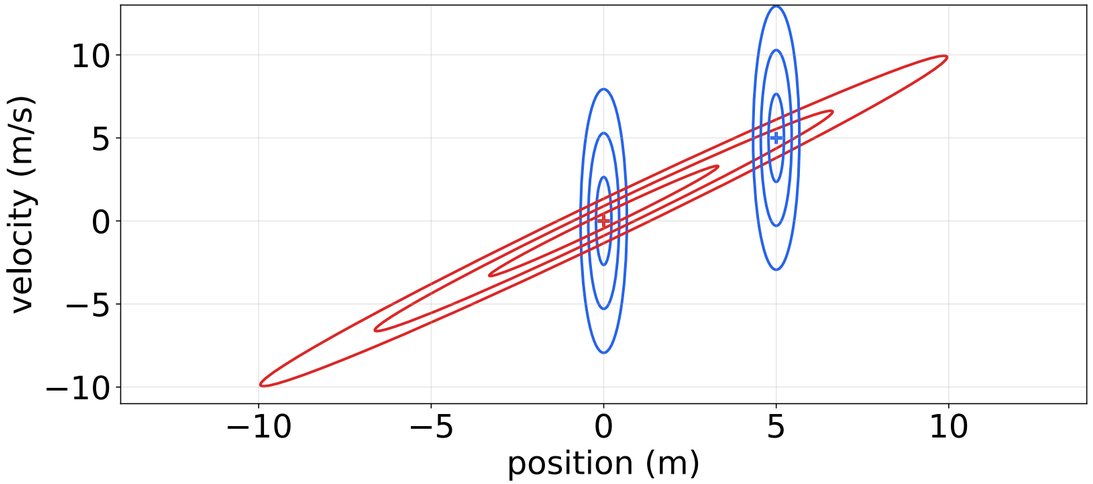


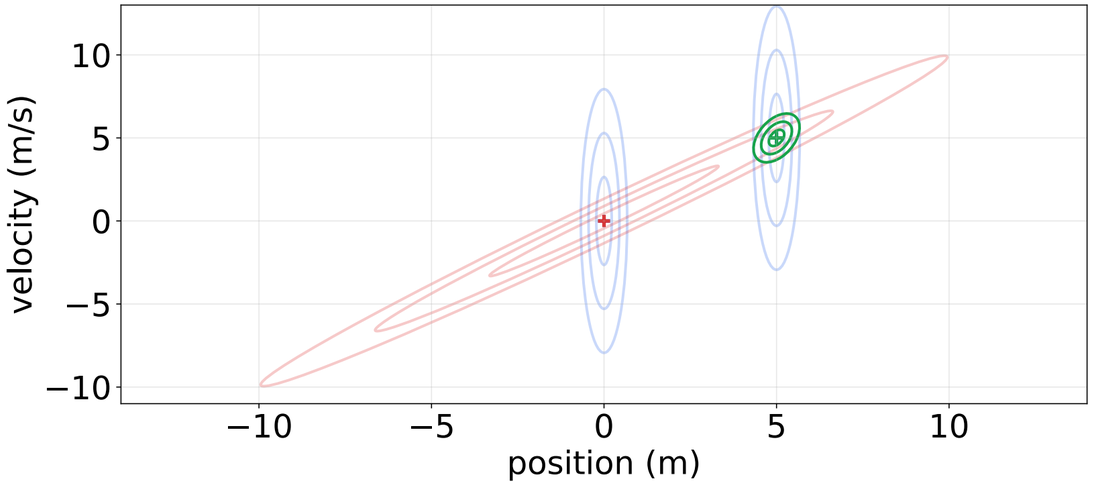


> **핵심**: 이 시점에서는 속도를 모르므로 속도 공분산만으로는 새 위치를 예측할 수 없음

Measurement and Posterior

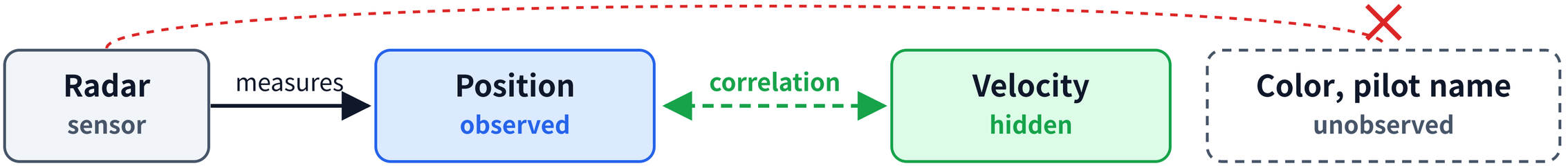
- **1 초 뒤 측정**: 위치 x = 5, 속도는 모름 → 세로 타원
- **합리적 추정**: 속도 공분산(빨강)과 측정의 교차 지점
- **곱한 결과(초록)**: 교차 지점, 약간 기울어 위치와 속도 상관
- **크기**: 훨씬 작음, x 폭도 측정보다 좁아 위치도 확실





> **핵심**: 서로 다르지만 상관된 두 정보를 곱하면 각각만 쓸 때보다 훨씬 정확한 추정

Observed, Hidden, Unobserved



velocity is inferred, not measured

Observed variable

- 레이더가 직접 측정하는 위치
- 측정마다 정보 직접 획득

Hidden variable

- 측정하는 센서가 없는 속도
- 위치와의 상관으로 매우 정확하게 추론

Unobserved variable

- 색, 조종사 이름, 엔진 RPM 등
- 측정도 상관도 없음, 상태에 넣으면 무의미한 추정

핵심: 두 측정 사이의 거리와 시간으로 속도를 대략 얻고, Bayes 정리로 몇 번의 관측 만에 정확한 추정

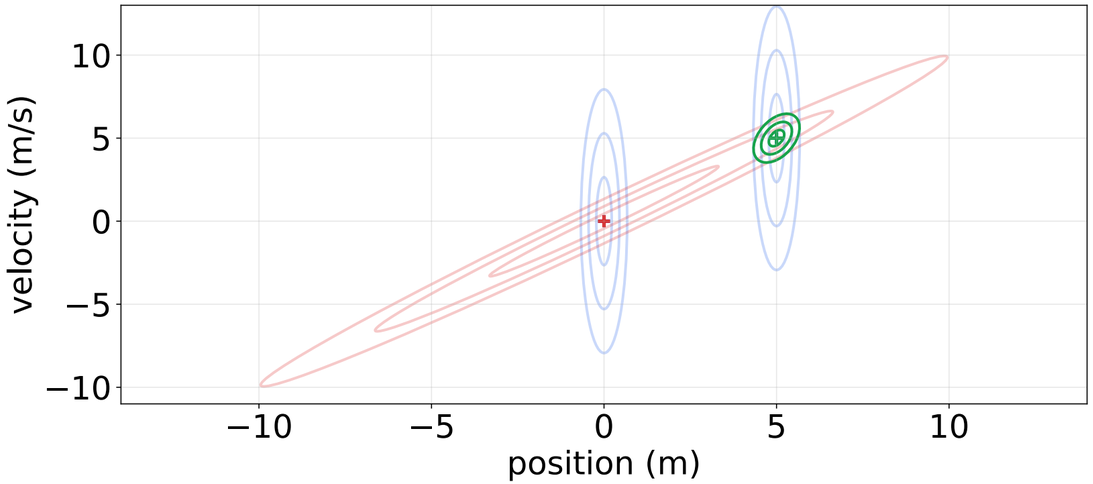
Why Correlation Helps

Unrelated velocity

- 다른 항공기의 속도를 위치 그래프에 겹치는 경우
- 두 정보가 무관하므로 결합해도 추가 정보 없음

Correlated velocity

- 이 항공기의 속도는 진행 방향과 빠르기를 알려 줌
- 속도가 바뀌면 위치도 예측 가능한 방식으로 바뀜
- 속도 변화와 맞지 않는 위치 측정은 참일 가능성이 매우 낮음



- **한 번 갱신**: 위치만 보면 속도를 포함한 효과가 작아 보임
- **여러 번 갱신**: 교차할 때마다 x 방향으로 좁아짐, 추정 분산 감소

> **핵심**: 위치와 속도의 상관을 이용하는 것이 다변량 Kalman filter 의 핵심 원리

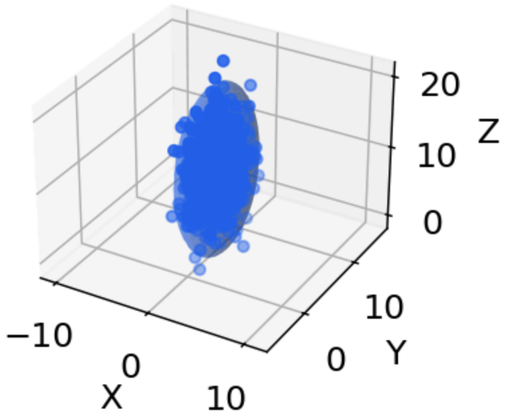
Higher Dimensions

- 2 차원 Gaussian 은 오차 타원, 3 차원 Gaussian 은 3D 오차 타원체(ellipsoid)로 표현
- 주어진 공분산의 표본 1000 개를 3σ 타원체와 함께 그리면 대부분의 점이 타원체 안에 위치
- 9 차원도 개념은 같음, 분포의 표준편차 오차를 9 차원 타원체로 기술, 일기 예보는 수천 차원
- **조건**: mu = [0.3, 5, 10], 공분산 C (분산 1, 4, 16.1), 표본 1000 개, std = 3

```python
from filterpy.stats import plot_3d_covariance

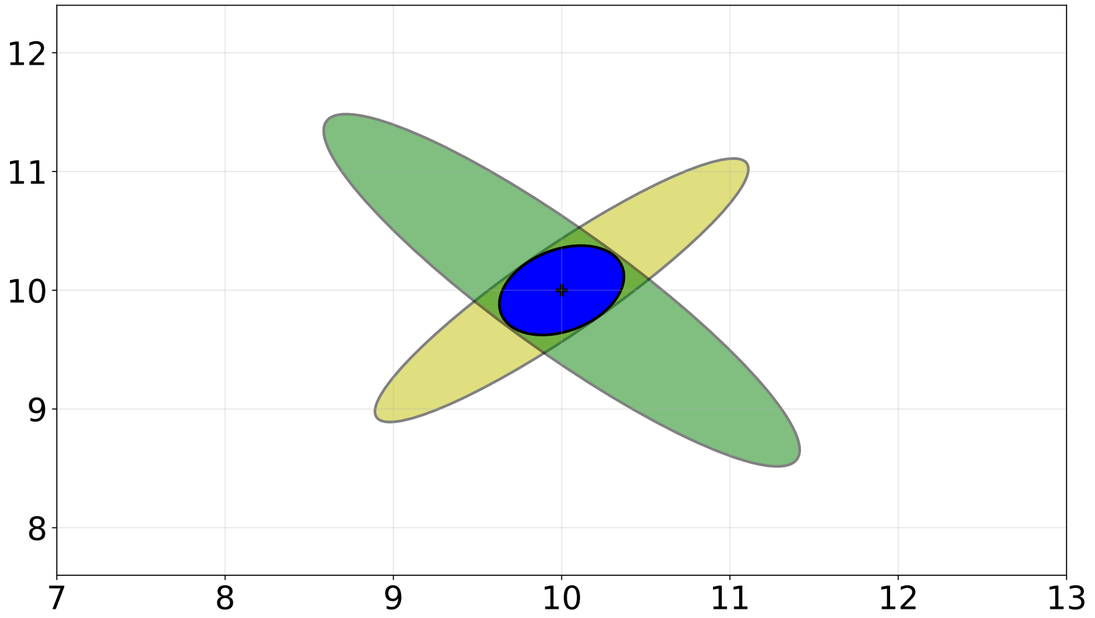
mu = [0.3, 5., 10.]
C = np.array([[1.0, .03, .2],
              [.03, 4.0, .0],
              [.2, .0, 16.1]])
sample = np.random.multivariate_normal(mu, C, size=1000)

ax = plot_3d_covariance(mu, C, alpha=.4, std=3,
                        limit_xyz=True)
ax.scatter(sample[:, 0], sample[:, 1], zs=sample[:, 2],);
```



Summary

- **표현**: 여러 변수의 믿음을 평균 벡터와 공분산 행렬의 다변량 Gaussian 으로
- **공분산**: 대각은 분산, 비대각은 공분산, 기운 타원은 상관
- **곱**: prior 와 likelihood 를 곱하면 더 정확한 posterior, 센서 기하가 반영된 삼각측량
- **hidden variable**: 측정하지 않는 속도를 위치와의 상관으로 추론
- **적용 범위**: 기하와 무관한 변수도 가능, 집 면적과 가격, 주가, 젖소의 우유 생산량



> **결론**: 불확실성을 상관을 가진 다차원 Gaussian 으로 표현하면 곱으로 훨씬 정확한 결과